

CS 151 Mancala Final Report

SP26 Section 01 Group 4

Zaeracami Carino, Edward Cui, Connor Petri

Github Link: <https://github.com/connor-petri/mancala-cs151>

Write Up for Design Pattern Assessment

Controller Class: MancalaController

Mutator method: pitClicked()

```
public MoveResult pitClicked(BoardModel.Player pitSide, int pitNum) {
    if (gameOver) {
        lastMoveResult = MoveResult.GAME_OVER;
        return MoveResult.GAME_OVER;
    }

    if (pitSide != currentPlayer) {
        lastMoveResult = MoveResult.INVALID_PIT;
        notifyListeners();
        return MoveResult.INVALID_PIT;
    }

    BoardModel.Pit pit = model.getPits(pitSide).get(pitNum);
    if (pit.numStones() == 0) {
        lastMoveResult = MoveResult.INVALID_PIT;
        notifyListeners();
        return MoveResult.INVALID_PIT;
    }

    undoManager.saveState(currentPlayer);
    // ...
}
```

Model Class: BoardModel

Mutator method: turn()

```
public boolean turn(Player player, int pitNum) {
    if (pitNum < 1 || pitNum > 6) {
        throw new IllegalArgumentException("Pit number must be between 1 and 6");
    }

    Pit current = switch (player) {
        case Player.ONE -> player1Pits.get(pitNum);
        case Player.TWO -> player2Pits.get(pitNum);
    };

    if (current.numStones() == 0) {
        throw new IllegalArgumentException("Pit " + pitNum + " is empty");
    }
}
```

```

    }

    // Save previous board state for undo
    p1StonesPrevious = new int[7];
    p2StonesPrevious = new int[7];
    for (int i = 0; i < 7; i++) {
        p1StonesPrevious[i] = player1Pits.get(i).numStones;
        p2StonesPrevious[i] = player2Pits.get(i).numStones;
    }

    // Turn mechanics
    int hand = current.numStones;
    current.numStones = 0;
    while (hand != 0) {
        current = current.next;
        boolean skip = (player == Player.ONE && current.equals(getBank(Player.TWO))
            || (player == Player.TWO &&
current.equals(getBank(Player.ONE))));
        if (!skip) {
            current.numStones++;
            hand--;
        }
    }

    // Capture mechanic
    System.out.println(current.name);
    if (current.adj != null && current.numStones == 1 && current.adj.numStones > 0
&& current.owner == player) {
        getBank(player).numStones += current.adj.numStones + 1;
        current.adj.numStones = 0;
        current.numStones = 0;
    }

    // Check for empty pits on either side
    boolean p1empty = true, p2empty = true;
    for (int i = 1; i < player1Pits.size(); i++) {
        if (player1Pits.get(i).numStones != 0) {
            p1empty = false;
        }
        if (player2Pits.get(i).numStones != 0) {
            p2empty = false;
        }
    }

```

```

    }

    if (p1empty) {
        for (int i = 1; i < player2Pits.size(); i++) {
            getBank(Player.TWO).numStones += player2Pits.get(i).numStones;
            player2Pits.get(i).numStones = 0;
        }
    }

    if (p2empty) {
        for (int i = 1; i < player1Pits.size(); i++) {
            getBank(Player.ONE).numStones += player1Pits.get(i).numStones;
            player1Pits.get(i).numStones = 0;
        }
    }

    // Return true if there is a free turn
    return current.equals(getBank(player));
}

```

View Class: BoardPanel

Notification method: refresh()

```

public void refresh() {
    BoardModel model = controller.getModel();

    for (int i = 1; i <= 6; i++) {
        pits.get("A" + i).setStoneCount(model.getPit("A" + i).numStones());
        pits.get("B" + i).setStoneCount(model.getPit("B" + i).numStones());
    }

    bank1Component.setCount(model.getBank(BoardModel.Player.ONE).numStones());
    bank2Component.setCount(model.getBank(BoardModel.Player.TWO).numStones());

    repaint();
    revalidate();
}

```

Strategy Interface: BoardStyle

```
public interface BoardStyle {  
    String getStyleName();  
    Color getFrameBackground();  
    Color getBoardBackground();  
    Color getPitColor();  
    Color getMancalaColor();  
    Color getStoneColor();  
    Color getLabelColor();  
    Font getLabelFont();  
    Color getTurnLabelColor();  
    Font getTurnLabelFont();  
    Color getStatusLabelColor();  
    Font getStatusLabelFont();  
    Color getButtonColor();  
    Color getButtonTextColor();  
    Color getButtonHoverColor();  
}
```

Concrete Strategies: ClassicBoardstyle & RetroBoardStyle

From StartScreenPanel (inside buildStyleSelection()):

```
classicButton.addActionListener(e -> selectedStyle = new ClassicBoardStyle());  
retroButton.addActionListener(e -> selectedStyle = new RetroBoardStyle());
```

From MancalaFrame:

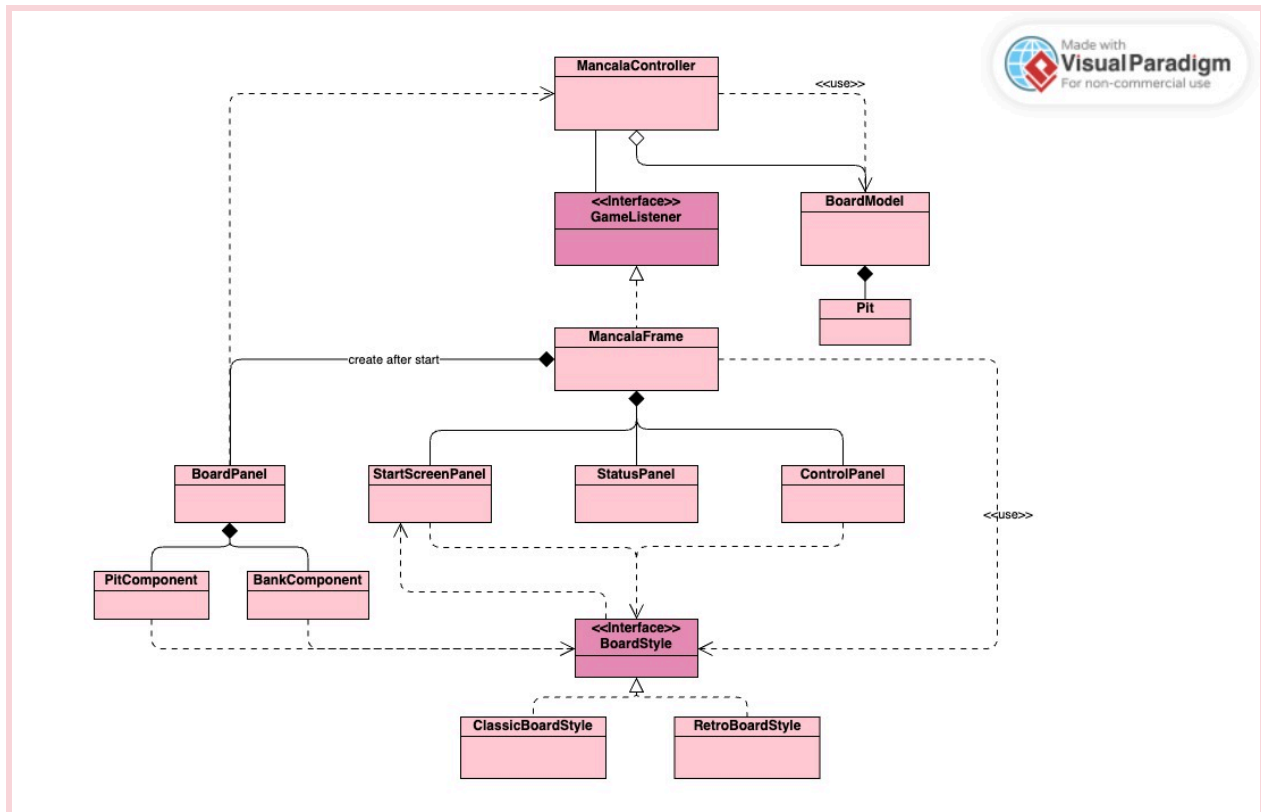
Inside wireStartScreen():

```
currentStyle = startScreenPanel.getSelectedStyle();
```

Inside showGameScreen():

```
boardPanel = new BoardPanel(controller, currentStyle);
```

Class Diagram



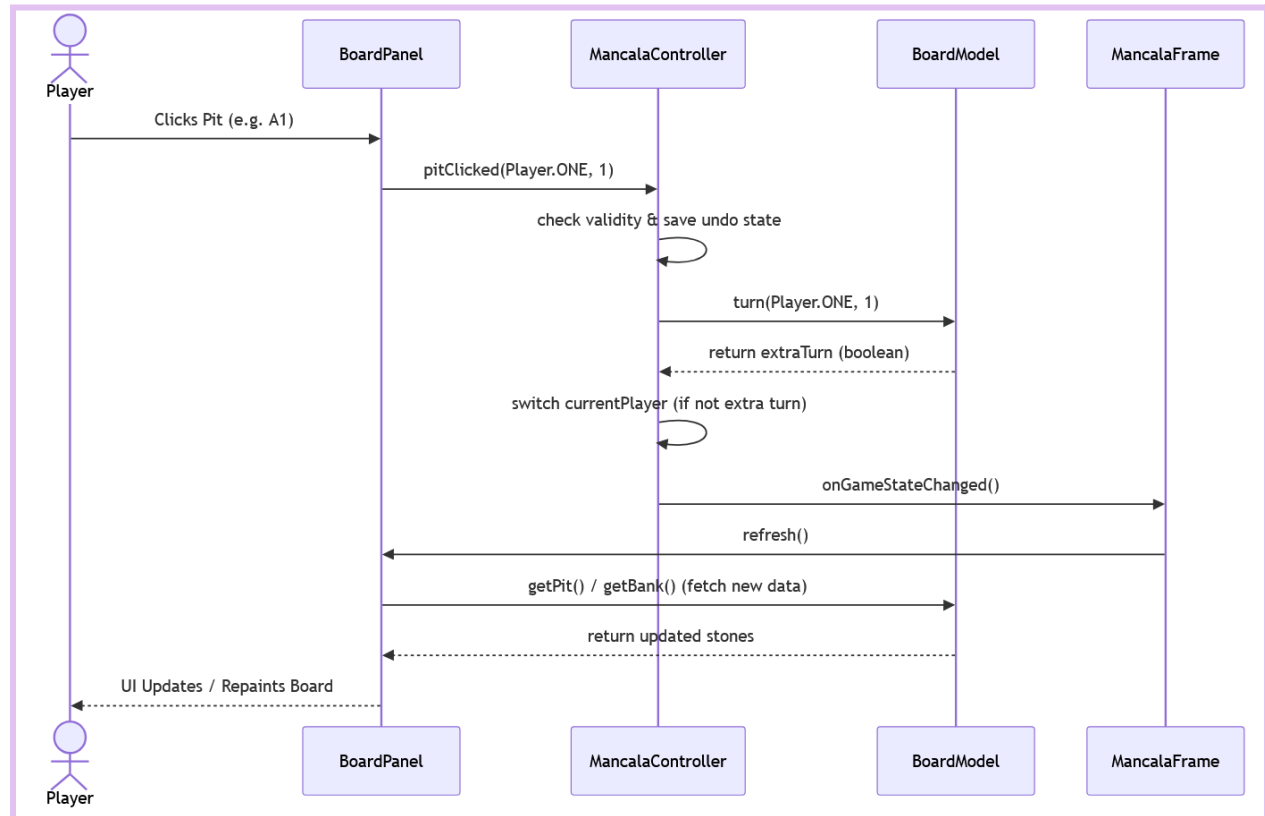
Use Cases

1. Start game
 - a. A player configures starting stones per pit and visual theme, and then starts a new game.
2. Play turn
 - a. A player selects a pit on their side. Stones within the pit are distributed counterclockwise.
 - b. The final stone may capture the stones in its opposite pit if the stone lands in an empty pit.
3. Undo move
 - a. A player undos their previous move, reverting the board to its pre-move state.
 - b. They can undo three times in a turn and only once in a row.
4. End game

- a. The game ends when all of a player's pits are empty and a winner is calculated.

Sequence Diagram

Play Turn Diagram



Reflection

Connor used MCV, aggregation, interfaces, interior classes, inheritance, and anonymous functions. He self-studied the rules of mancala and collaborative git (i.e. merging, pull requests).

Cami used encapsulation, inheritance, interfaces, composition, MVC, the Strategy pattern and Swing GUI development. She self-studied how to implement hover effects on buttons and the drop shadow effect for the start screen panel and improved layout to better align and center components. She also explored creating custom-shaped UI elements by overriding paintComponent(), such as oval pits, but chose not to implement them due to added complexity.

Edward used observer patterns, class design, and the MVC architecture pattern. He self studied a bit into state management for the undo system, how to make a proper game-based event listener system, as well as the mancala empty pit capture rule.